

OlimpiaPeru: Sistema Web de Gestión de Eventos Deportivos Universitarios

Avance de Proyecto Final 3 — Unidad 3: Introducción al JavaScript

Wilder Esteban Abanto Garcia — Código U23310744

Equipo: 404 not found (modalidad individual)

Universidad Tecnológica del Perú

Taller de Programación Web - Sección 20953

Docente: Kelvin Celso Macedo Ylachoque

5 de julio de 2026

1. Introducción

OlimpiaPeru es un sitio web que centraliza la organización y difusión de las olimpiadas deportivas interfacultades de la Universidad Tecnológica del Perú (UTP). El proyecto nació como una página estática construida con HTML5 y CSS3 y, a lo largo del curso, ha incorporado de manera progresiva las tecnologías revisadas en cada unidad del sílabo.

El presente informe corresponde al Avance de Proyecto Final 3 (semana 15) y documenta la incorporación de JavaScript al proyecto, conforme al temario de la Unidad 3: variables y constantes, estructuras de control, arreglos y operadores, métodos de entrada y salida del navegador (prompt, confirm, alert), manipulación del DOM y construcción de un menú responsivo con ventanas flotantes. A diferencia de los avances anteriores, en los que el sitio era completamente estático, esta entrega transforma a OlimpiaPeru en una aplicación interactiva del lado del cliente.

2. Descripción del Avance

2.1 Evolución del proyecto

Hasta el Avance de Proyecto Final 2, OlimpiaPeru funcionaba como un sitio de una sola página (index.html) con navegación por anclas, en el que el formulario de inscripción y la tabla de posiciones eran completamente estáticos. En este tercer avance, seis funcionalidades del sitio pasan a depender de JavaScript para su comportamiento: la validación del formulario de inscripción, el registro dinámico de resultados en la tabla de posiciones, el sorteo automático de series de competencia, el menú de navegación responsivo, el resaltado de la sección activa durante el scroll y el carrusel de la sección de inicio.

2.2 Alcance de este avance

El alcance de este avance comprende exclusivamente la capa de comportamiento (JavaScript) del sitio. No se modificó la estructura semántica de las secciones definida en el Avance 1 (HTML) ni el sistema de diseño definido en el Avance 2 (CSS); el trabajo de esta unidad se concentra en dotar de interactividad a los componentes que antes eran estáticos, sin backend ni persistencia de datos en servidor: todo el estado (equipos, resultados, validaciones) vive en memoria del navegador durante la sesión de uso.

3. Objetivos

3.1 Objetivo general

Implementar el comportamiento dinámico de OlimpiaPeru utilizando JavaScript del lado del cliente, aplicando de forma coherente los conceptos de variables, estructuras de control, arreglos, manipulación del DOM y eventos revisados en las sesiones 11 a 14 del curso.

3.2 Objetivos específicos

- Validar el formulario de inscripción en el navegador mediante expresiones regulares y manipulación del DOM, sin depender únicamente de los atributos nativos de HTML5.
- Estructurar los datos del torneo en arreglos de objetos JavaScript y generar la tabla de posiciones dinámicamente a partir de dichos datos.
- Aplicar los métodos de entrada y salida del navegador (prompt, confirm, alert) en un flujo de sorteo de series que dependa de la interacción del usuario.
- Construir un menú de navegación responsivo controlado enteramente por JavaScript, reemplazando la técnica de checkbox oculto en CSS puro usada en el avance anterior.
- Implementar una ventana flotante (modal) para mostrar resultados generados dinámicamente, sin recargar la página.

- Incorporar retroalimentación visual en tiempo real sobre la sección visible de la página mediante manipulación de clases del DOM mientras el usuario navega.

4. Tecnologías y Fundamento de Decisiones

La siguiente tabla describe cada técnica de JavaScript aplicada en este avance, su función dentro del proyecto y el motivo de su elección.

Tecnología / Técnica	Qué hace en el proyecto	Por qué se eligió
Expresiones regulares (RegExp)	Valida el formato del código UTP (U + 8 dígitos) y del correo institucional (@utp.edu.pe) en formulario.js.	Permite validar patrones específicos de la institución que HTML5 no puede expresar con sus atributos nativos.
Manipulación del DOM (document)	Crea, elimina y modifica elementos (mensajes de error, filas de tabla, bloques de resultados) en tiempo real.	Es la base de la Unidad 3; permite reaccionar a la interacción del usuario sin recargar la página.
Arreglos de objetos	Modela a los equipos del torneo como un arreglo de objetos {nombre, g, e, p} en tabla.js, en vez de filas fijas en el HTML.	Separa los datos de su representación visual, permitiendo ordenar, actualizar y agregar equipos con métodos de arreglo (sort, push, find).
Métodos de entrada/salida del navegador	prompt() solicita el número de series a sortear, confirm() valida la operación antes de ejecutarla y alert() informa errores de validación (equipos insuficientes, número fuera de rango).	Corresponde directamente al temario de la sesión 12 y ofrece una forma simple de capturar entrada del usuario sin construir un formulario adicional.
Algoritmo Fisher–Yates (Durstensfeld)	Mezcla aleatoriamente el arreglo de equipos antes de repartirlos en series, dentro de sorteo.js.	Es el algoritmo estándar para lograr una permutación aleatoria uniforme en tiempo $O(n)$, evitando sesgos frente a un ordenamiento por Math.random() ingenuo.
Eventos del DOM (addEventListener)	Escucha clics (botones de resultado, sorteo, menú), envío de formulario, teclas (Escape) y cambios de input.	Es el mecanismo estándar para separar la lógica de comportamiento del marcado HTML (sin atributos onclick en línea).
IntersectionObserver API	Detecta qué sección del sitio está visible durante el scroll y resalta el enlace correspondiente en el menú (scrollspy.js).	Es más eficiente que calcular posiciones de scroll manualmente en cada evento scroll, ya que el navegador gestiona la detección de forma asíncrona.

classList y atributos ARIA	Alterna la clase open del menú y actualiza aria-expanded/aria-label en el botón hamburguesa (menu.js).	Mantiene el menú accesible para lectores de pantalla al reflejar su estado real, no solo su apariencia visual.
----------------------------	--	--

5. Funcionalidades JavaScript Implementadas

A continuación se describe cada funcionalidad incorporada en este avance, con un fragmento representativo del código fuente ubicado en la carpeta js/ del repositorio.

4.1 Validación del formulario de inscripción (formulario.js)

El envío del formulario se intercepta con preventDefault() y se valida cada campo con funciones propias: nombre no vacío, código UTP con el patrón U#####, correo institucional @utp.edu.pe, contraseña de al menos 6 caracteres, categoría y disciplina seleccionadas, y aceptación de términos. Cada error se muestra como un elemento insertado junto a su campo, y se elimina automáticamente en cuanto el usuario corrige el valor.

```
const codigoUTP = /^[Uu]\d{8}$/;
const correoUTP = /^[^\s@]+@utp\.edu\.pe$/i;

form.addEventListener("submit", function (e) {
  e.preventDefault();
  const errores = validar();
  if (errores.length > 0) {
    errores.forEach(pintarError);
    return;
  }
  // ... muestra mensaje de éxito y limpia el formulario
});
```

Figura 1. Validación del formulario con expresiones regulares y manejo del evento submit.

4.2 Tabla de posiciones dinámica (tabla.js)

Los cinco equipos del torneo se representan como un arreglo de objetos con sus partidos ganados, empatados y perdidos. La función render() ordena el arreglo por puntaje ($g \times 3 + e$) y reconstruye las filas de la tabla en el DOM, incluyendo el resaltado del top 3 con una medalla. Los botones de

resultado permiten sumar un partido ganado, empatado o perdido al equipo seleccionado, y el formulario de la sección permite agregar nuevos equipos sin editar el HTML.

```
const equipos = [
  { nombre: "Ingeniería de Sistemas", g: 4, e: 1, p: 0 },
  { nombre: "Administración de Empresas", g: 3, e: 1, p: 1 },
  // ...
];

function render() {
  equipos.sort((a, b) => puntos(b) - puntos(a) || b.g - a.g);
  tbody.innerHTML = "";
  equipos.forEach((t, i) => { /* construye <tr> y lo agrega al DOM */ });
}
```

Figura 2. Modelo de datos en arreglo y renderizado dinámico de la tabla de posiciones.

4.3 Sorteo automático de series con ventana flotante (sorteo.js)

Al presionar el botón de sorteo, el script lee los nombres de equipo directamente desde las filas actuales de la tabla, pide con `prompt()` el número de series deseado, confirma la operación con `confirm()` y valida los límites con `alert()` en caso de error. Los equipos se mezclan con el algoritmo de Fisher–Yates y se reparten en series mediante round-robin; el resultado se pinta dentro de una ventana modal que puede cerrarse con un botón, haciendo clic fuera de ella o presionando la tecla `Escape`.

```
const entrada = prompt("¿En cuántas series repartir los " + n + " equipos?");
const series = parseInt(entrada, 10);
if (isNaN(series) || series < 2 || series > n) {
  alert("Ingresa un número válido de series.");
  return;
}
if (!confirm("Se sortearán " + n + " equipos en " + series + " series. ¿Continuar?"))
  return;
mezclar(equipos); // Fisher–Yates
```

Figura 3. Flujo de entrada/salida del navegador (prompt, confirm, alert) en el sorteo de series.

4.4 Menú de navegación responsivo (menu.js)

El menú hamburguesa, que en el Avance 2 se controlaba con un checkbox oculto y selectores CSS, se reemplazó por un botón real controlado por JavaScript. Al hacer clic se alterna la clase `open` del

<nav>, se actualizan los atributos aria-expanded y aria-label, y el ícono cambia entre barras y una X. El menú también se cierra automáticamente al seleccionar cualquier opción de navegación.

```
function alternar() {
  const abierto = nav.classList.toggle("open");
  btn.setAttribute("aria-expanded", abierto);
  icono.className = abierto ? "fa-solid fa-xmark" : "fa-solid fa-bars";
}
btn.addEventListener("click", alternar);
```

Figura 4. Menú hamburguesa controlado por JavaScript, con estado reflejado en atributos ARIA.

4.5 Resaltado de sección activa al hacer scroll (scrollspy.js)

Un IntersectionObserver observa cada sección del sitio y activa el enlace de navegación correspondiente cuando esa sección cruza la franja central de la ventana visible. Este mecanismo reemplazó la marca fija en 'Inicio' que tenía el menú antes de este avance, y no requiere calcular manualmente la posición de scroll en cada evento.

```
const observer = new IntersectionObserver((entradas) => {
  entradas.forEach((e) => {
    if (e.isIntersecting) {
      enlaces.forEach((a) => a.classList.remove("active"));
      mapa[e.target.id].classList.add("active");
    }
  });
});
}, { rootMargin: "-45% 0px -50% 0px" });
```

Figura 5. Uso de IntersectionObserver para el resaltado dinámico del menú de navegación (Mozilla Contributors, 2026).

6. Estructura de Archivos Actualizada

La lógica que antes vivía en un único script.js se organizó en módulos independientes dentro de la carpeta js/, replicando el criterio de separación por funcionalidad que ya se aplicaba en css/.

Archivo	Responsabilidad
js/slider.js	Autoplay del carrusel de inicio, navegación con flechas y puntos, y pausa al pasar el cursor.
js/formulario.js	Validación completa del formulario de inscripción y mensajes de error/éxito.
js/tabla.js	Modelo de datos de los equipos, ordenamiento por puntos y renderizado de la tabla de posiciones.

js/sorteo.js	Lectura de equipos desde el DOM, mezcla Fisher–Yates, reparto en series y ventana modal de resultado.
js/menu.js	Apertura/cierre del menú responsivo y sincronización de atributos ARIA.
js/scrollspy.js	Resaltado del enlace de navegación correspondiente a la sección visible.
js/himno.js	Control del reproductor de audio nativo de la sección Multimedia.

En index.html, los ocho módulos se enlazan con etiquetas `<script>` planas al final del documento, sin bundler ni sistema de módulos, en línea con el enfoque introductorio de la Unidad 3.

7. Relación con el Sílabo — Unidad 3

El siguiente cuadro vincula cada sesión de la Unidad 3 (semanas 11 a 14) con su evidencia concreta en el proyecto.

Sesión	Tema del sílabo	Implementación en OlimpiaPeru
11	Introducción a JavaScript: conceptos, variables, constantes, implementación con HTML y ejecución en consola.	Declaración de variables y constantes en todos los módulos de js/ (const form, const equipos, let series, etc.) y uso de la consola del navegador para depuración durante el desarrollo.
12	Estructuras de control, arreglos y operadores. Métodos de entrada y salida: prompt, confirm y alert.	Condicionales y bucles en formulario.js y tabla.js; arreglo de objetos equipos; prompt/confirm/alert en el flujo completo de sorteo.js.
13	Manipulación del HTML con JS: introducción al DOM, manipulación del DOM con document.	Creación y eliminación de elementos (mensajes de error, filas de tabla, bloques de series) con document.createElement, appendChild e innerHTML controlado.
14	Creación de un menú responsivo y ventanas flotantes con HTML, CSS y JS.	Menú hamburguesa controlado por JS (menu.js) y ventana modal de resultados del sorteo (sorteo.js + css/modal.css).

8. Bitácora de Desarrollo

El proyecto mantiene un registro interno de cambios (docs/logbook/changelog.md), del cual se extraen las tareas correspondientes a este avance:

Código	Tarea	Resumen
--------	-------	---------

RM-001	Validación del formulario con JavaScript	Validación por DOM: bloquea el envío inválido, muestra mensajes por campo y un banner de éxito.
RM-002	Tabla de posiciones dinámica	Los equipos pasan a un arreglo JS; se agregan controles para registrar resultados y nuevos equipos con re-render automático.
RM-006	Separación de script.js por funcionalidad	El script monolítico se divide en los módulos listados en la sección 6, sin bundler ni módulos ES.
RM-003	Sorteo automático de series con ventana flotante	Lectura de equipos del DOM, prompt/confirm/alert, mezcla Fisher–Yates y modal de resultado.
RM-005	Menú responsivo controlado por JavaScript	Migración de checkbox+label a botón real con classList.toggle, ícono dinámico y aria-expanded.
RM-008	Scrollspy en el nav	IntersectionObserver resalta la sección visible durante el scroll, reemplazando la marca fija en Inicio.

Las fechas de finalización de cada tarea se conservan en el archivo original del repositorio del proyecto.

9. Trabajo Pendiente

El registro de deuda técnica del proyecto (docs/logbook/technical-debt.md) mantiene abierta una observación de riesgo bajo relacionada con contenido de relleno aún presente en index.html: una imagen de marcador de posición (picsum.photos), una pista de audio de demostración (SoundHelix) y un párrafo de presentación institucional genérico. Estos elementos no afectan el funcionamiento de las tecnologías evaluadas en esta unidad, pero quedan identificados como pendientes a reemplazar por contenido propio antes de la entrega del Proyecto Final.

10. Conclusiones

Este avance incorpora a OlimpiaPeru el conjunto de temas de la Unidad 3: variables y constantes, estructuras de control, arreglos, manipulación del DOM y construcción de un menú responsivo con ventana flotante. La introducción de JavaScript transformó el sitio de una página estática a

una aplicación interactiva del lado del cliente, sin perder la estructura semántica ni el sistema visual desarrollados en las unidades anteriores.

La separación del código en módulos independientes (uno por funcionalidad) facilitó mantener cada script enfocado en una sola responsabilidad, siguiendo el mismo criterio de organización ya aplicado a las hojas de estilo. El uso de arreglos de objetos para modelar los datos del torneo, en lugar de filas fijas en el HTML, permitió que la tabla de posiciones y el sorteo de series operen sobre una única fuente de verdad. Con este avance, el proyecto queda listo para la etapa de integración final de HTML, CSS y JavaScript prevista en la última unidad del curso.

Referencias

- Durstenfeld, R. (1964). Algorithm 235: Random permutation. Communications of the ACM, 7(7), 420. <https://doi.org/10.1145/364520.364540>
- Lerma-Blasco, R. (s. f.). Aplicaciones web. Universidad de La Sabana. <https://tubiblioteca.utp.edu.pe/cgi-bin/koha/opac-detail.pl?biblionumber=30954>
- Mozilla Contributors. (2026, 1 de marzo). Intersection Observer API. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/API/Intersection_Observer_API
- Pavón Puertas, J. (2015). Creación de un sitio web con PHP y MySQL. RA-MA Editorial. <https://elibro.net/es/ereader/utpbiblio/106491>
- Universidad Tecnológica del Perú. (2026). Sílabo: Taller de Programación Web - Sección 20953, Ciclo 1 Marzo.